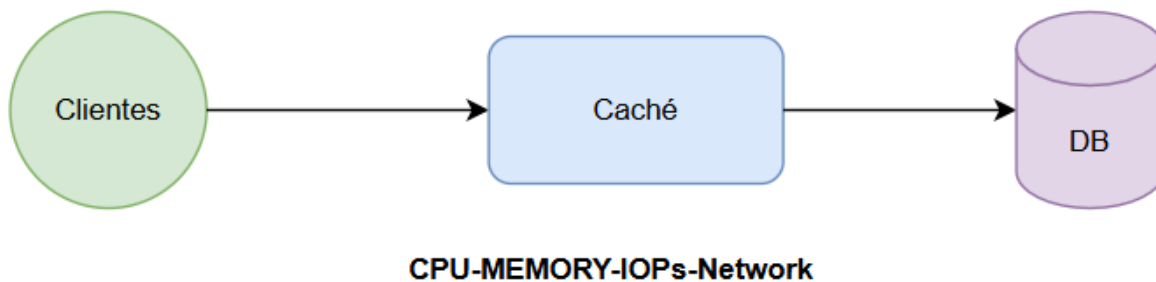


# Cachés

---

Al trabajar con grandes volúmenes de usuarios y peticiones, es importante reducir la carga sobre la base de datos.

Por ejemplo, si tengo 1 millón de usuarios leyendo un periódico, cada uno haría una llamada a la base de datos, lo cual generaría un uso muy alto de recursos como CPU, memoria, red, IOPs y ancho de banda. Para evitar esto, se introduce una caché entre los usuarios y la base de datos.



## ¿Qué es una caché?

Una caché no necesariamente es solo espacio en memoria, pero en el contexto de bases de datos sí suele ser un espacio de memoria considerable.

- Almacena información en forma de **Key-Value pairs** (pares clave-valor).
- Los datos tienen un **TTL (Time to Live)** que define cuánto tiempo permanecen en la caché.
- Implementa un **algoritmo de reemplazo**, el cual define la política para decidir qué elementos eliminar cuando la caché se llena.

---

## Políticas de reemplazo

Cuando la caché alcanza su límite, se debe aplicar una política de reemplazo. La política a utilizar depende del funcionamiento y del caso de uso específico. Las más comunes son:

- **FIFO (First In, First Out)**: Se elimina el primer elemento que entró.
- **LIFO (Last In, First Out)**: Se elimina el último elemento que entró.
- **MRU (Most Recently Used)**: Se elimina el elemento más recientemente utilizado.
- **LFU (Least Frequently Used)**: Se elimina el elemento menos utilizado.

---

## Almacenamiento de Key-Value Pairs en Cachés

---

Los key-value pairs se almacenan generalmente aplicando un hash sobre la clave. Un ejemplo común es usar un **MD5** sobre la clave de la consulta entrante.

El cálculo del hash genera una cadena de tamaño fijo. Esta cadena funciona como una tabla de indexamiento,

lo cual brinda muchas ventajas en la rapidez de acceso. El tiempo de respuesta de una caché es bastante bajo en comparación con acceder directamente a la base de datos.

**md5**(SELECT \* FROM articles WHERE date = today) = **FE567398ABC (\*)**

---

## Cómo se realiza un mapeo?

Se puede imaginar una **hash table** con múltiples espacios para almacenar claves y valores. Cuando llega una

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |
| 9 | A | B |
| C | D | E |
|   |   |   |

clave, se pasa por una **función hash**.

Por ejemplo:

- Usar el módulo 15 (**key % 15**).
- El resultado define el bucket (campo en tabla) donde se almacena o consulta el valor. Con esto, la búsqueda es sumamente rápida.

En términos prácticos, una caché funciona como una tabla hash que permite localizar valores en el menor tiempo posible.

- También es común utilizar árboles en cachés para organizar y acceder a los datos. Una práctica habitual es guardar en caché las consultas más utilizadas por los clientes, lo que reduce el tiempo invertido en procesarlas nuevamente.

## Funcionamiento de las Cachés en la Vida Real

---

En la práctica, las cachés aparecen en múltiples escenarios de la computación. Generalmente, se parte de una relación entre dos recursos:

- **Un recurso rápido, pero caro.**

- **Un recurso lento, pero barato.**

El objetivo de la caché es equilibrar las velocidades entre ambos, evitando la subutilización del recurso caro (que implica pérdida de dinero).

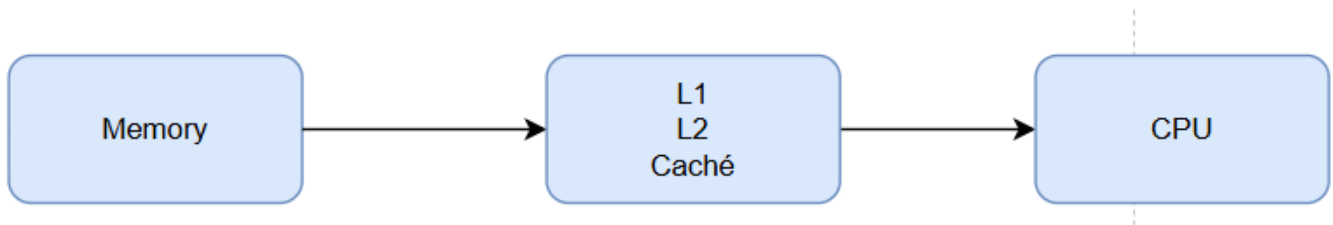


---

## Algunos ejemplos

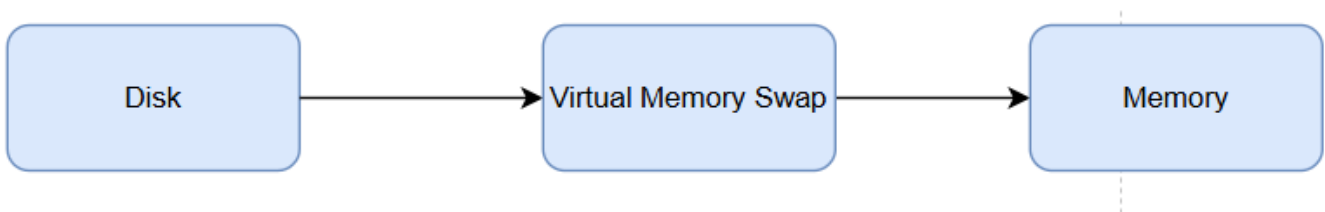
### 1. CPU y memoria

- La CPU es extremadamente rápida en comparación con la memoria principal.
- Para equilibrar esto, se utilizan cachés L1 y L2, que reducen la brecha de velocidad.



### 2. Memoria y disco

- La memoria es mucho más rápida que el disco duro.
- Para mejorar el rendimiento, se utiliza memoria virtual o swap.
- El disco guarda páginas de memoria formateadas que luego son leídas por la memoria, ahorrando tiempo de conversión de formato.



### 3. Bases de datos y clientes

- Acceder directamente a la base de datos es muy costoso en términos de recursos. Para reducir este costo, se introducen cachés como Memcached, Redis o Druid.io.
- En arquitecturas modernas (apps móviles, web o desktop), el flujo suele ser: **Cliente** → **API** → **Caché** → **Base de datos** → **Red**

---

## Cachés en la red

- La red está limitada por el bandwidth (ancho de banda), que es sumamente caro.

- Si los clientes entran directamente a los servidores, los componentes empiezan a fallar debido a:
  - Sobrecarga de usuarios.
  - Fallos en la base de datos.
  - Saturación de recursos.

Para esto se implementan CDN (Content Delivery Networks) o cachés web mediante redes de distribución de contenido. Estas funcionan como cachés que replican información en diferentes ubicaciones geográficas.

- Ventajas:
  - Reducen el consumo de ancho de banda del servidor.
  - Mejoran la velocidad de acceso al acercar los datos a los usuarios.
  - Disminuyen el round trip time (RTT), es decir, el tiempo que tarda una consulta en llegar al servidor y regresar.

De esta forma, los clientes reciben los datos más rápido y sin necesidad de acceder directamente al servidor de origen.

## Diferentes Arquitecturas de Motores de Bases de Datos

---

### PostgreSQL

PostgreSQL ofrece múltiples sistemas de replicación y arquitecturas que permiten adaptarse a distintos escenarios.

---

### Arquitectura: Single Database

La arquitectura Single Database consiste en tener Un solo disco y una sola base de datos. Además, una instalación que incluye el software de la BD, el sistema operativo y las librerías necesarias para ambos.

#### Ventajas

- Existe un solo reloj (CPU), lo que simplifica la sincronización.
- No es necesario utilizar la red para coordinar operaciones.
- Las operaciones se ejecutan de manera atómica dentro de un ciclo de reloj.
- Permite implementar las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

#### Desventajas

- Se cae fácilmente en un Single Point of Failure (punto único de falla).
- Reduce la disponibilidad del sistema.

## TipoS de Arquitectura

---

### File System Replication

La File System Replication funciona de la siguiente manera:

- Existe una instalación de la base de datos con un proceso llamado **postgresql** (el sistema corriendo en la computadora). Este proceso trabaja en memoria.
- Los datos se almacenan en un file system (similar a carpetas en Windows o Linux).
  - En Linux, una ubicación típica es:

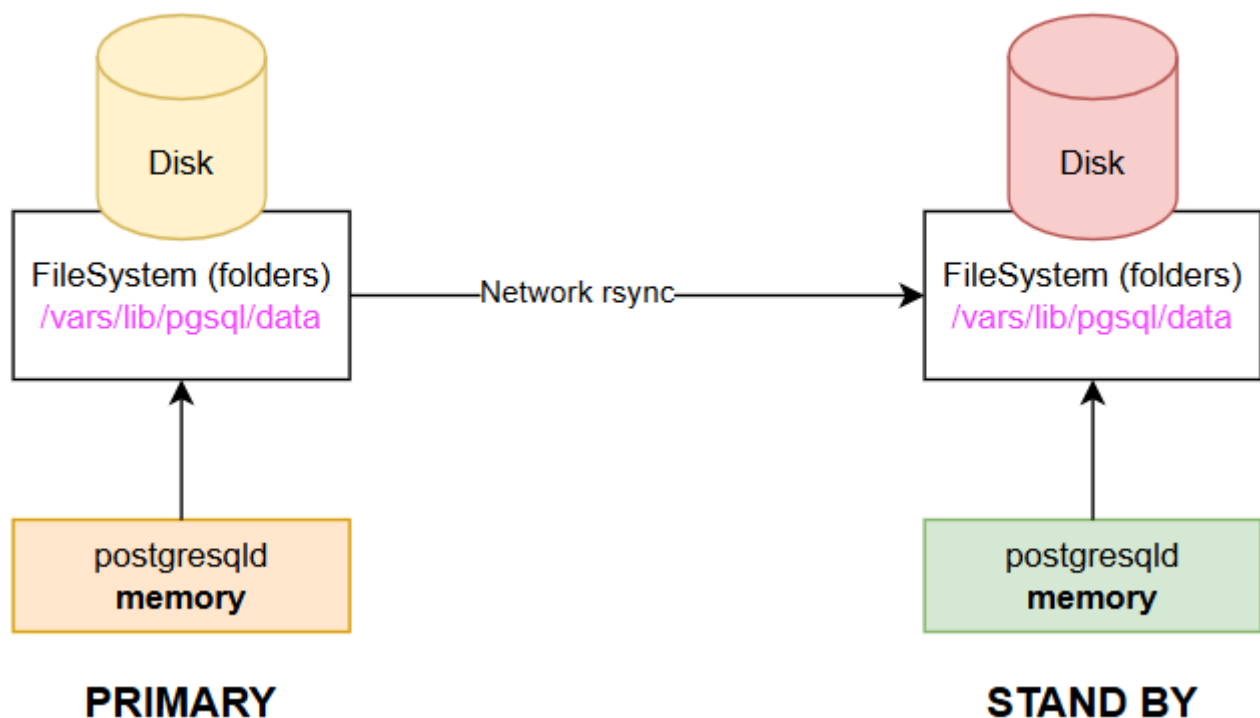
```
/var/lib/pgsql/data
```

- Allí se guardan datos, índices e información relacionada.

Después, siempre existe una BD en vivo (primaria) y una BD en modo réplica (slave). La sincronización se realiza a nivel de file system, entonces:

- Cuando llega una transacción al disco del primario, esta se copia a través de la red al disco del servidor réplica.
- En caso de fallo del primario, se puede realizar un switchover o failover (manual o automático).
- Las aplicaciones se redirigen para comunicarse con el servidor que queda como primario.

De esta forma, se asegura cierta continuidad operativa aunque la sincronización ocurre a nivel de disco y depende de la red.



## Shared Disk Failover

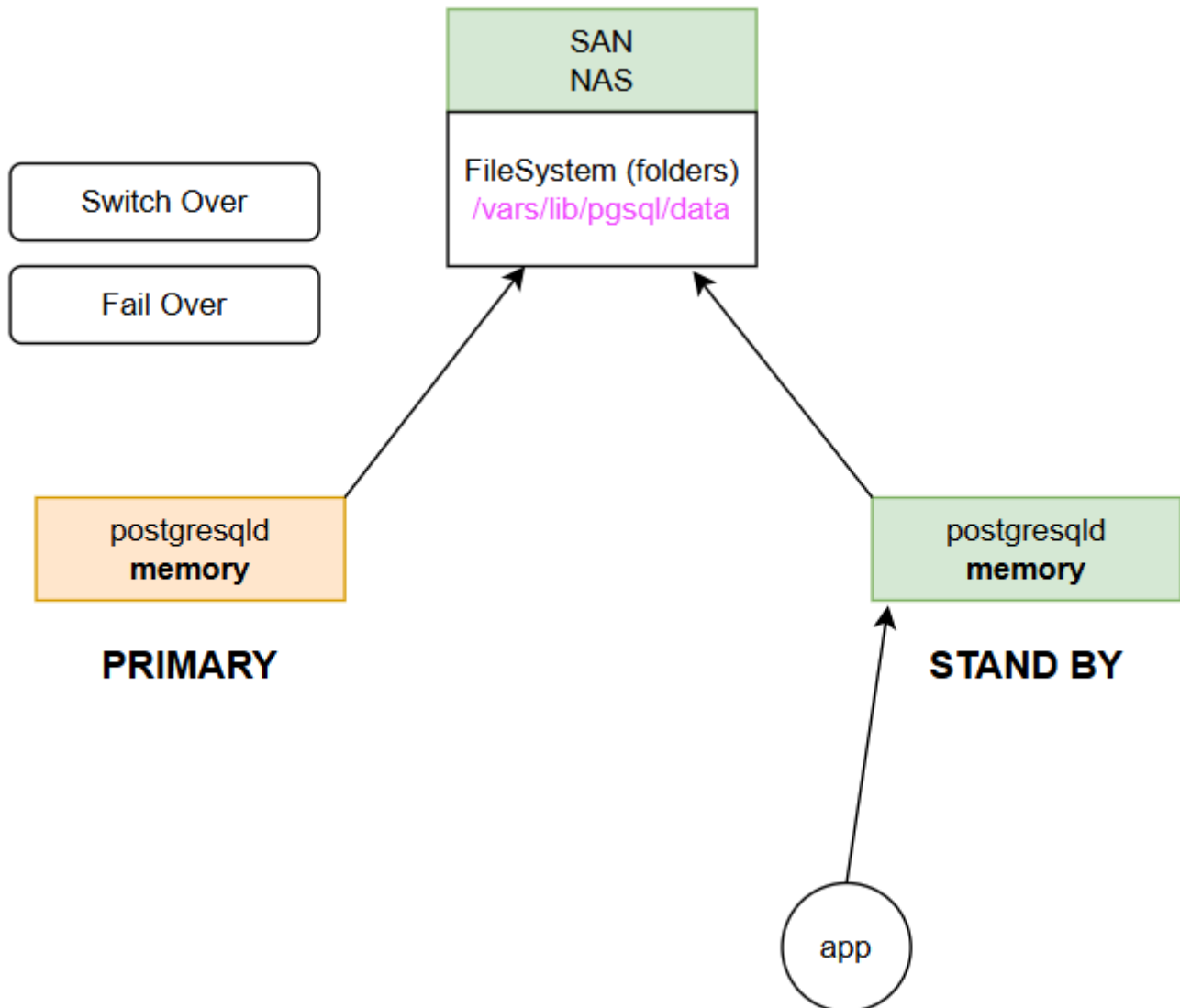
En la arquitectura Shared Disk Failover se utilizan sistemas de almacenamiento como SAN (Storage Area Network) o NAS (Network Attached Storage).

*¿Cómo funciona?*

- Existen dos instancias de la base de datos: una primary y una standby. Tanto la instancia primaria como la standby acceden al mismo sistema de archivos, ya que el SAN/NAS permite que ambos vean los

mismos datos.

- La información se almacena en el file system compartido.
- Cuando un servidor falla, el otro puede levantarse y continuar la operación utilizando los mismos datos. En este modelo se comparte un disco duro o lógico entre dos o más servidores, lo que asegura que los datos sigan disponibles.
- También existe un mecanismo de **switchover o failover** que le indica a la aplicación cuál servidor debe utilizar.



## Write-Ahead Log (WAL) Shipping

También se puede decir que es el corazón de la Base de Datos para cumplir con ACID. El Write-Ahead Log (WAL), también conocido como binary log o log shipping, es el mecanismo central que permite garantizar las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

El WAL es un archivo secuencial (*append-only*).

Características:

- Escrituras rápidas.
- Nunca se sobrescriben posiciones previas: siempre se escribe en la última posición disponible.
- Está presente tanto en bases de datos de un solo nodo como en arquitecturas de múltiples nodos.

## Ejemplo de transacción

Se tienen las siguientes operaciones:

```
UPDATE balance = 100 WHERE name = "Nereo";  
UPDATE balance = 0 WHERE name = "Marlon";
```

Es importante recalcar que el software de la BD siempre en memoria principal tendrá un espacio con Data files e Index files. La BD siempre intenta mantener todo en memoria para evitar tener que usar memoria virtual o ir directamente a disco. En memoria se cargan pequeñas porciones de data files e index files.

Entonces cuando llega un tipo de transacción así el flujo es:

- Si la transacción entra, se coloca en el data file, pero aún no llega a disco.
- Si se escribiera directamente a disco, sería sumamente lento, porque implicaría acceso constante al disco.
- Se escribe la transacción en el binary log.
- Se asigna un número a la transacción.
- Mediante una operación append, se escribe en el binary log.
- En ese momento, la transacción es recibida y registrada como una escritura secuencial de tipo append

**T6543: BEGIN**

**T6543: UPDATE balance = 100 WHERE name = 'nereo'**

**T6543: UPDATE balance = 0 WHERE name = 'marlon'**

- Después de eso la bd viene y actualiza el data file y procede a decirle al cliente que ya todo ha sido exitoso
- Cuando la bd termina de hacer la actualización en memoria, estos cambios van a meterse en un buffer, este buffer es un espacio en memoria donde vamos a monitorear si el dato se ha escrito a disco o no.
- En el momento que la bd termina, se hace otra entrada en el binary log diciendo **commit** que significa que ha sido escrita a disco.

```
T6543: BEGIN
T6543: UPDATE balance = 100 WHERE name = 'nereo'
T6543: UPDATE balance = 0 WHERE name = 'marlon'
T6543: COMMIT
```

